BIL 222 System Programming-Spring 2024

Ammar Daşkın

Hours

Fridays at 2:30pm at B517, slides and class examples are posted on classroom

Thursdays at 3:30pm at B517

A hw/quiz will be given on github.com or in class with different tools

Prerequisites

A passing grade from BIL 121 C programming

A little data structures, algorithms, and computer architecture

Resources

Textbooks and Course Material

- Lectures follow the book on the link https://github.com/angrave/SystemProgramming/wiki
- https://man7.org/index.html by Michael Kerrisk.
- For C programming,
  - you can refer to the book "The C Programming Language", Brian W Kernighan, Dennis M Ritchie,
  - BIL121 lecture notes: https://sites.google.com/view/adaskin/teaching/c-programming-lecture-notes

Other Books

- More complete guide
  - The Linux Programming Interface: A Linux and UNIX System Programming Handbook, by Michael Kerrisk, https://man7.org/index.html by Michael Kerrisk.
- Just for the topics in this course
  - Linux System Programming: Talking Directly To The Kernel And C Library Second Edition by Robert Love
- Advanced Programming in the UNIX Environment 3rd Edition, by W. Richard Stevens

Grading policy

10% Weekly Labs/Quizzes

- Each quiz covers the topics in the previous lecture
- 1 lowest grade is not considered in the final grade
- Given through **google classroom or github**

30% 3-4 programming assignments (including group projects)

20% midterm exam

40% final exam

How to study this course?

Weekly lecture shows theory and simple implementations of the topic

➔ You may need to take notes, some concepts or details may not be in the slides.

Labs, you practice and review the topics discussed in the lectures,

➔ some topics that are skipped in the lectures may be included in the lab write-ups.

In hw and projects, you complete the implementations discussed in the lectures or implement some algorithms/concepts discussed in the lectures.

➔ Function prototypes and algorithms are given or described.
➔ Submitted through github and classroom.

What is this course about?

Basics of system programming

- the practice of writing system software by **using core system libraries**

System softwares interfaces with the OS-kernel

- Shell
- Text editor
- Debugger
- System daemons
- Network server
- Database server

Why system programming?

The majority of operating systems are written at the system level

Knowledge of system programming provides you to understand how to write low level interface programs and write better high level code

System programming in Linux

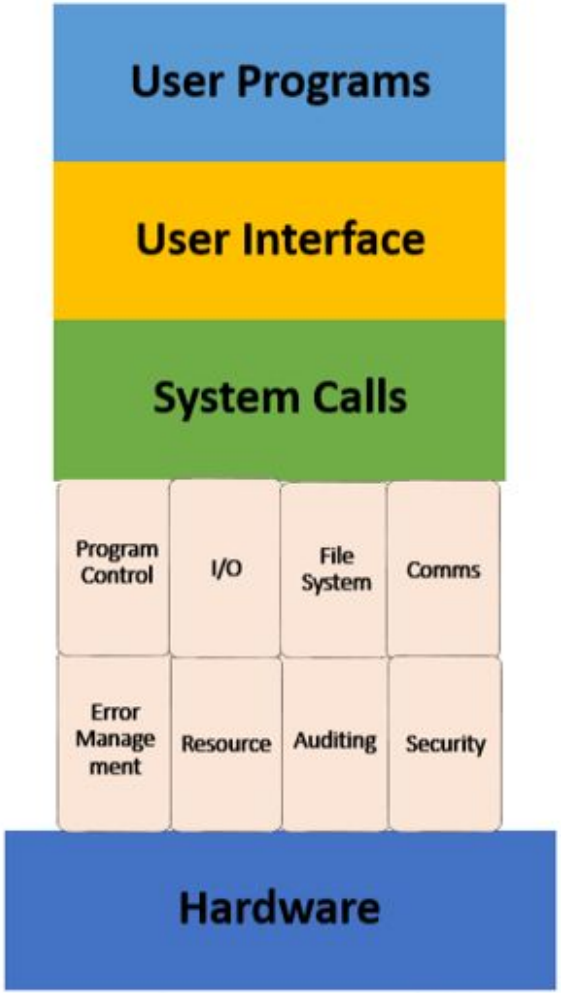
System calls

- Function invocations made from user space into the kernel
- Kernel provides a mechanism for user space applications to **signal** the kernel to invoke a **system call**

C library

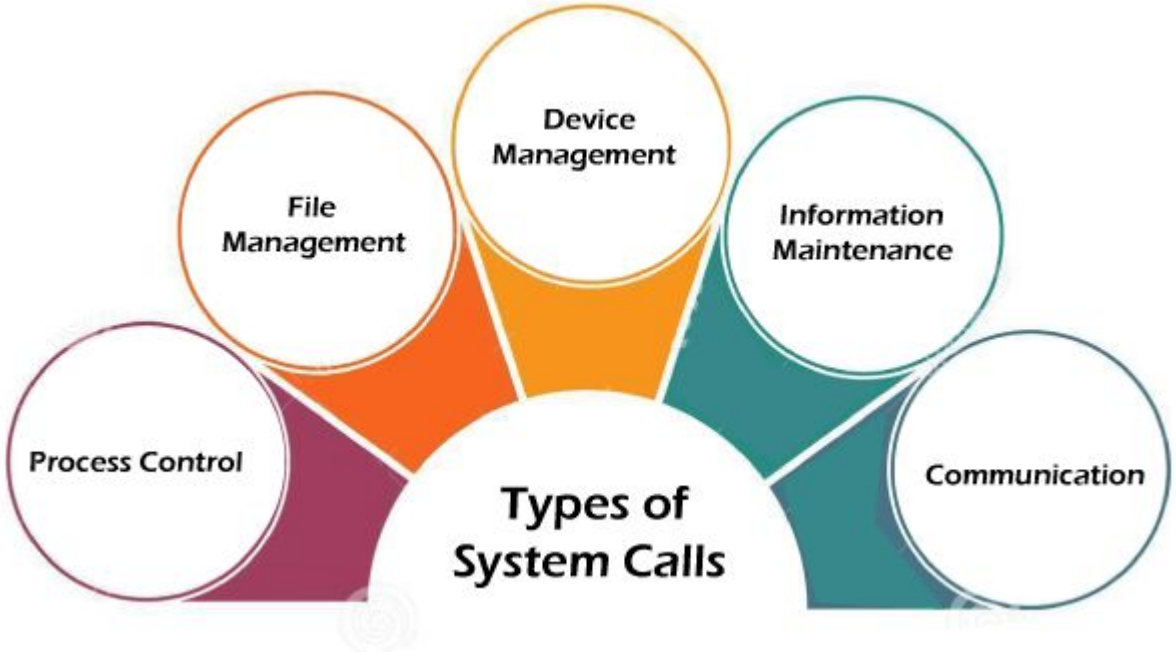
Standard C compiler is provided by GNU compiler collection (gcc), GNU version of cc

Computer OS Structure



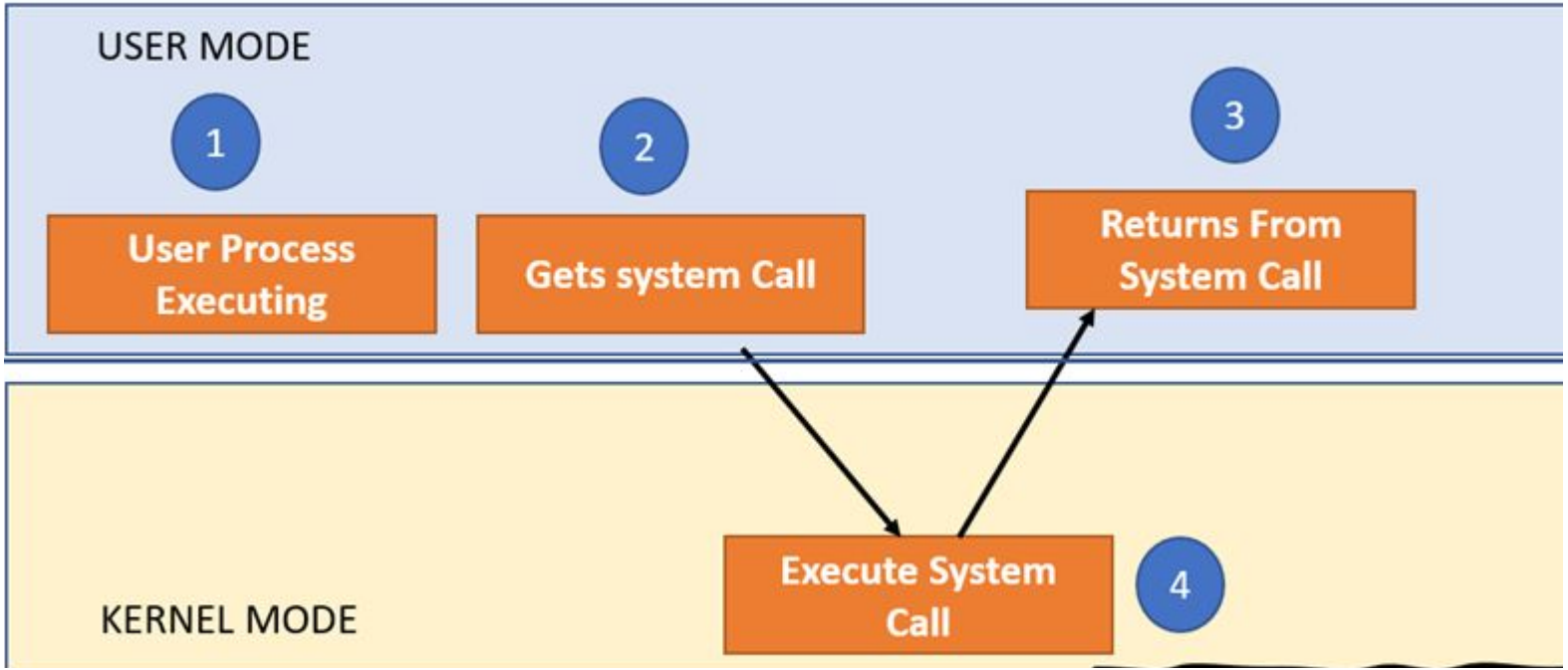
https://www.guru99.com/system-call-operating-system.html#2

What type of system calls we can use in our system programs



https://www.javatpoint.com/system-calls-in-operating-system

OS services the system calls!



https://www.guru99.com/system-call-operating-system.html#2

For portability

API (application programming interface)

- Defines interfaces by which one software can be invoked from another software
- Generally between a low level software and a high level software (API-user)
  - Example: Interfaces defined by C library for memory management and string manipulations

ABI

- Binary interfaces between two software pieces on particular architecture
- Guarantees that a piece of software function on any system with the same ABI

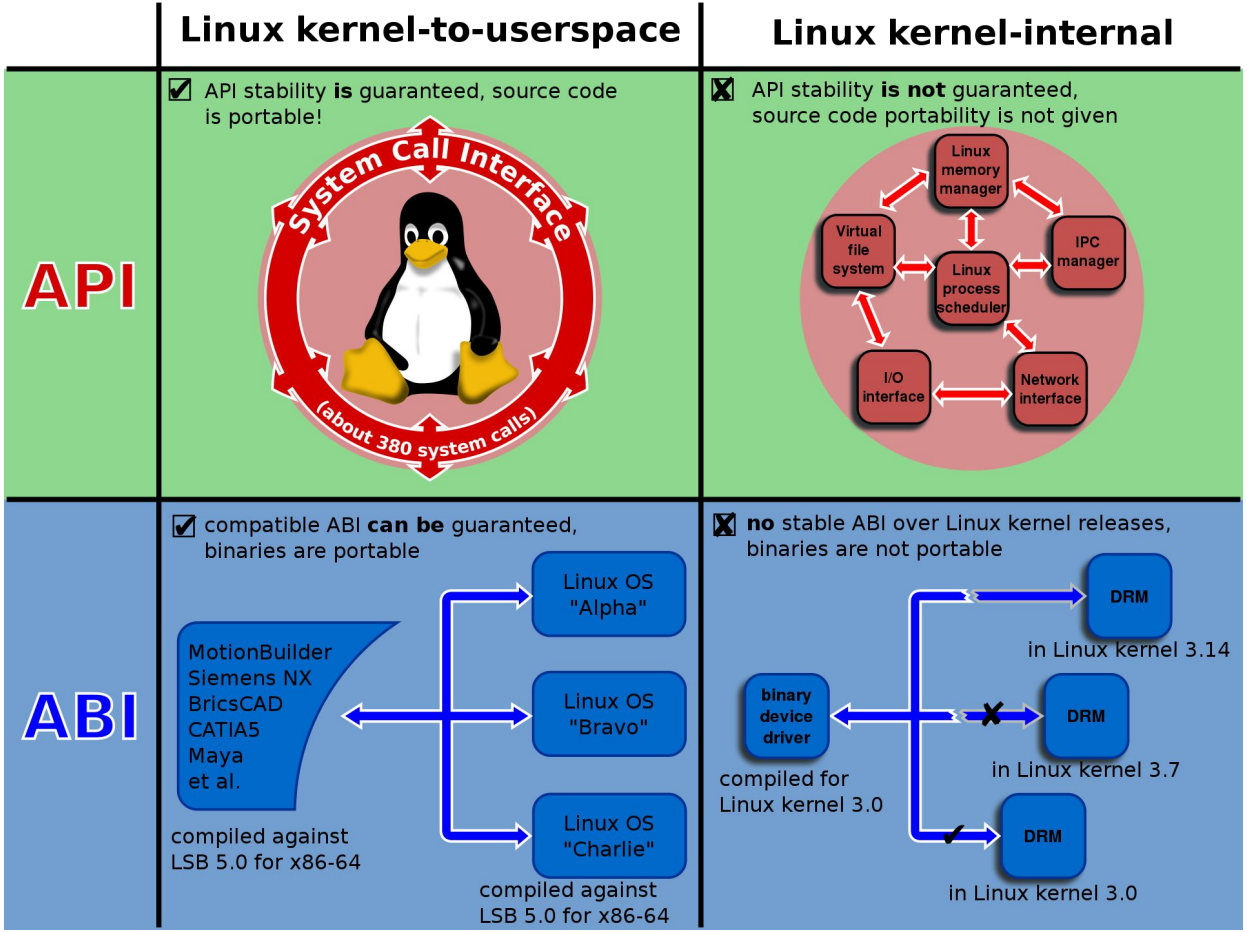
Compliance

Linux aims to comply with two of the POSIX and the Single UNIX Specification (SUS) standards.

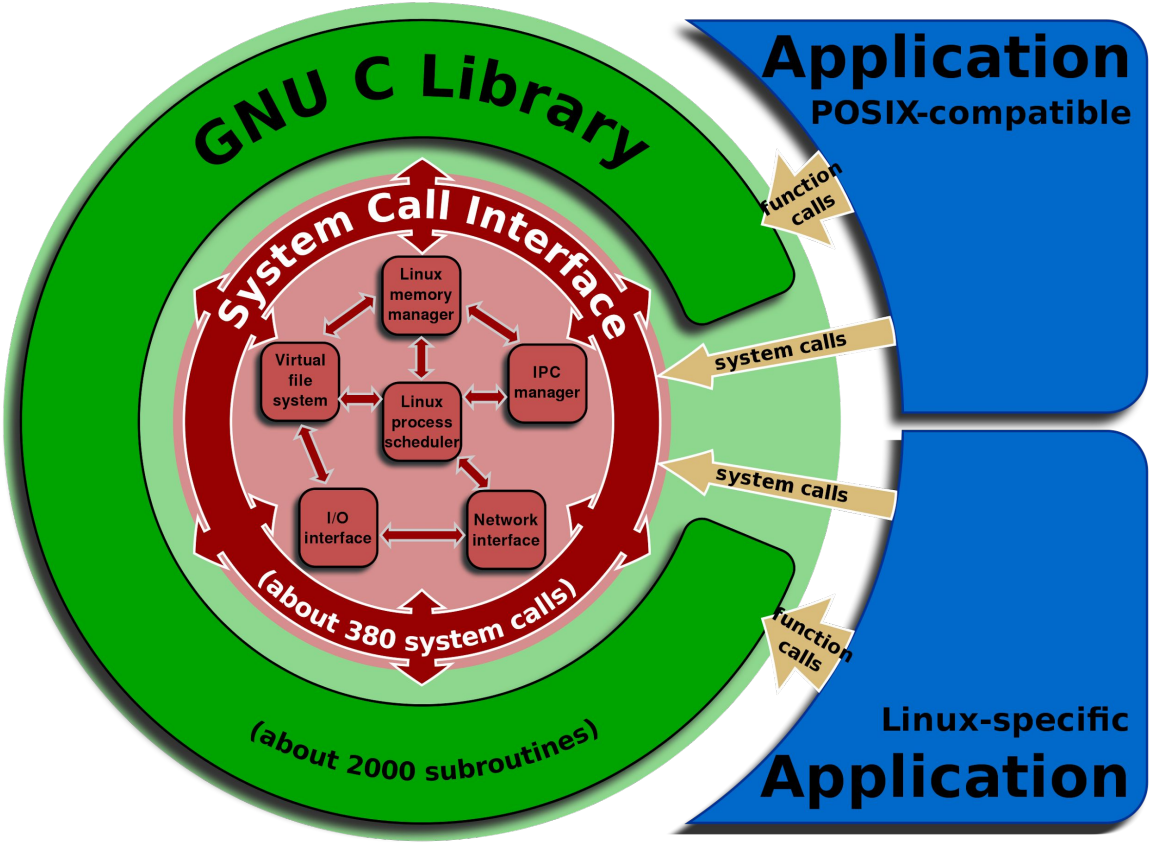
POSIX (Portable Operating System Interface)

- Suggested by Richard Stallman, founder of free software development
- Issued by IEEE in 1988 and revised on later years
- Many system calls and interfaces are standardized by POSIX

C Language standards: gcc compiler is ISO C99-compliant



https://en.wikipedia.org/wiki/System_call



https://en.wikipedia.org/wiki/Linux_kernel_interfaces#Linux_API

Weekly Content

1. Course administrivia, git, gcc, gdb, crash course on C
2. Review of pointers, C memory model, makefiles
3. Processes
4. Memory allocators
5. Threads
6. Synchronization
7. Midterm review
8. Deadlock
9. Virtual memory & IPC
10. Scheduling
11. Networking-Socket programming
12. Filesystems
13. Signals
14. Final-Review

Git intro

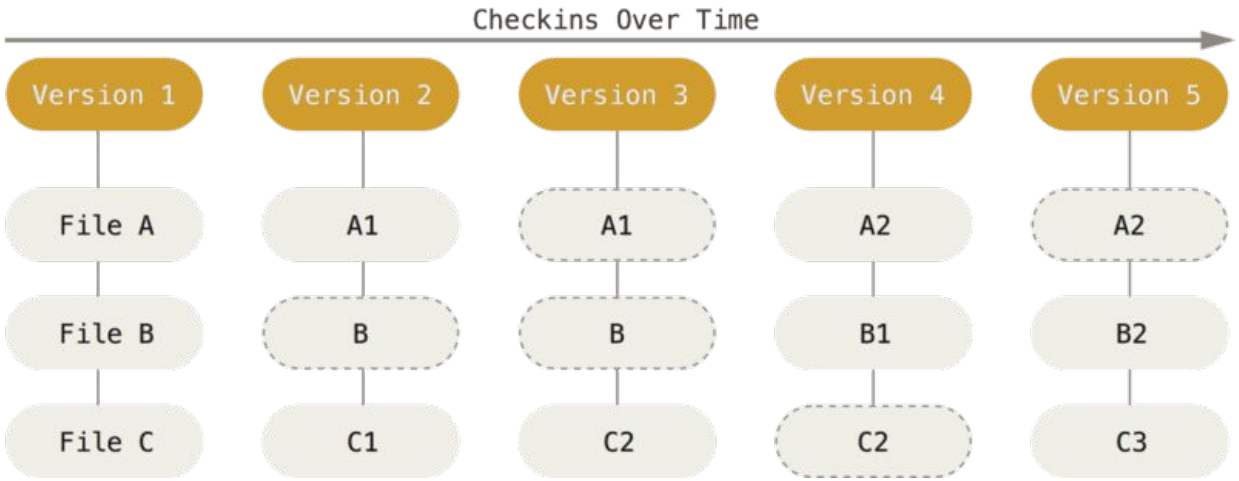
This semester we will use github.com for assignments…

Github uses **git** version control system

So what is git?

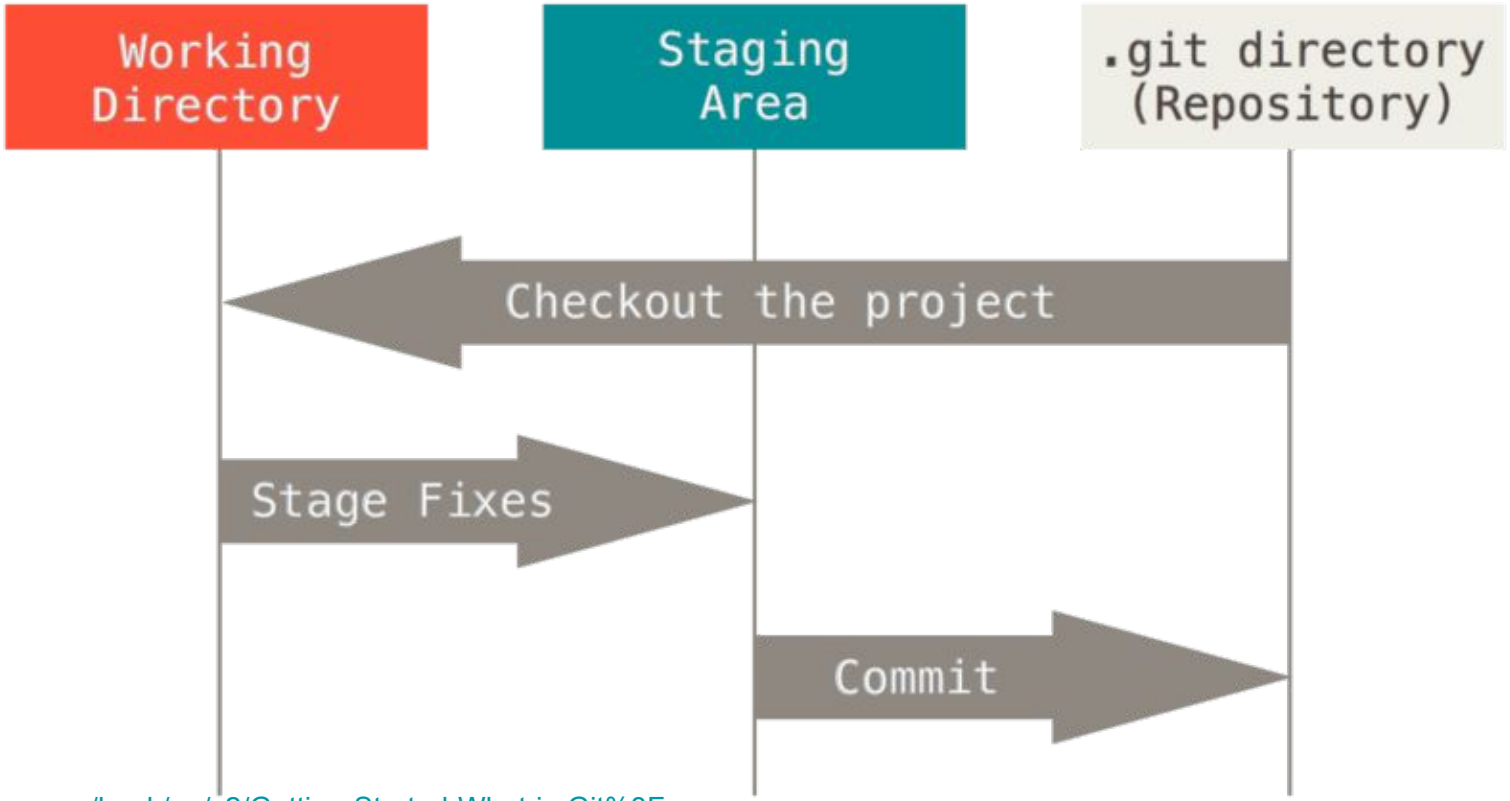
A version control system such as CVS, subversion, Perforce

- It stores an entire directory-called **repository**



https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F

Git states (modified, staged, and committed)



https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F

Initializing a repository in an existing directory

for Linux:

```
$ cd /home/user/my_project
```

for macOS:

```
$ cd /Users/user/my_project
```

for Windows:

```
$ cd C:/Users/user/my_project
```

and type:

$ git init

This creates a new subdirectory named `.git` that contains all of your necessary repository files

Begin tracking existing files

```
$ git add *.c
```

```
$ git add LICENSE
```

do an initial commit

```
$ git commit -m 'Initial project version'
```

https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository#ch02-git-basics-chapter

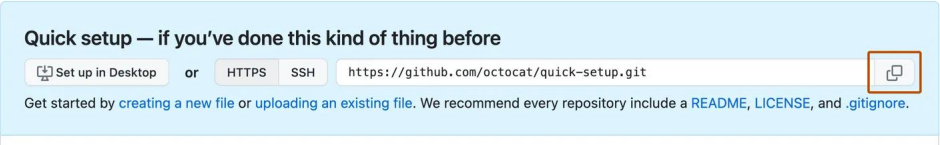
To add this to github

$git init -b main

$ git add .

$ git commit -m "First commit"

Create new repo on github.com




```
$ git remote add origin REMOTE-URL
```

to check if url is correct

```
$ git remote -v
```

Push changes to github.com

```
$ git push origin main
```

- You have submitted your assignment

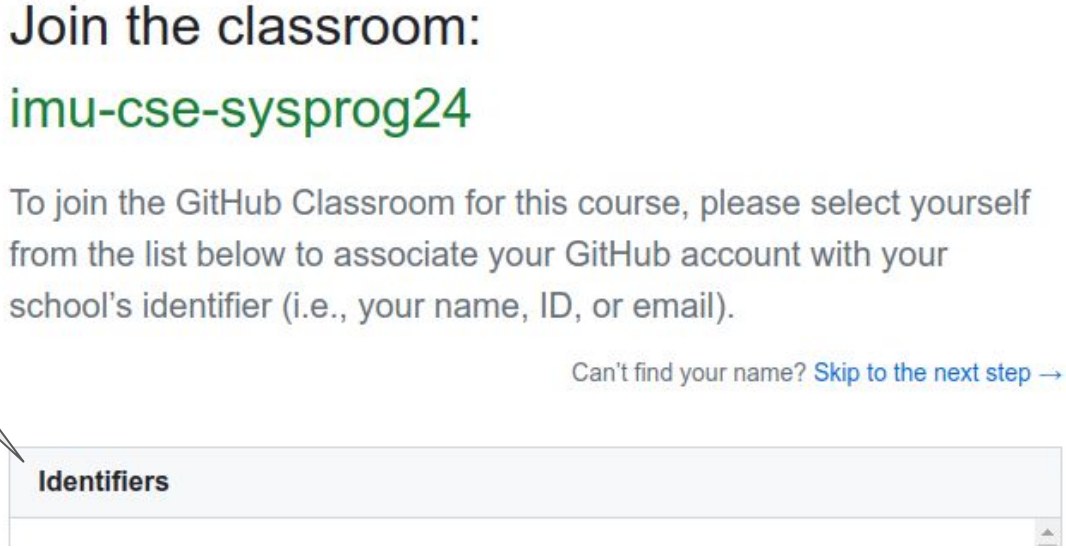
See for windows, macos, and linux
https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github

For assignments

Use the given assignment link on your browser

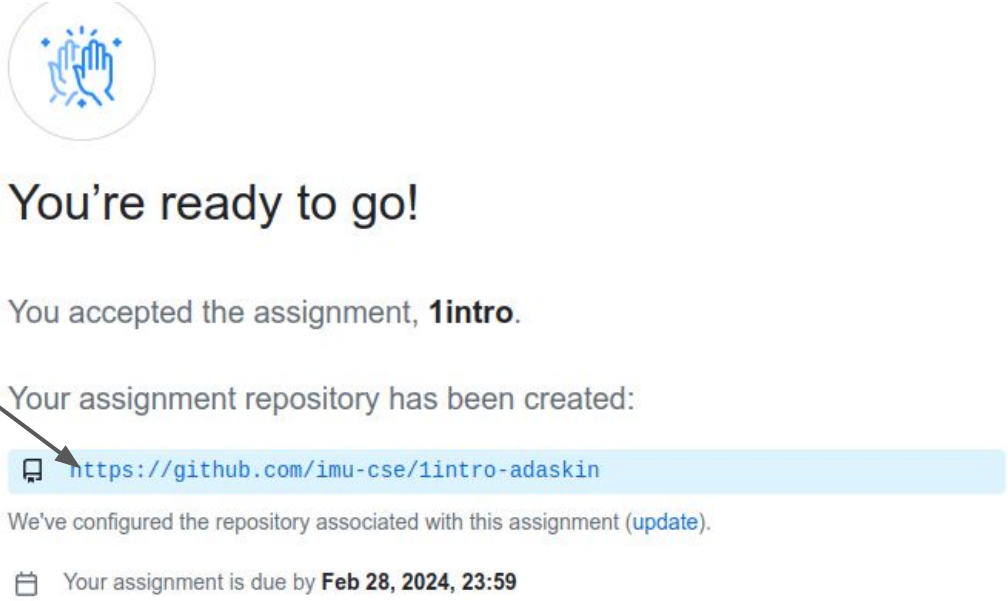
https://classroom.github.com/a/?????….

Choose your name in **identifiers** to link your github account



One you accepted an assignment,

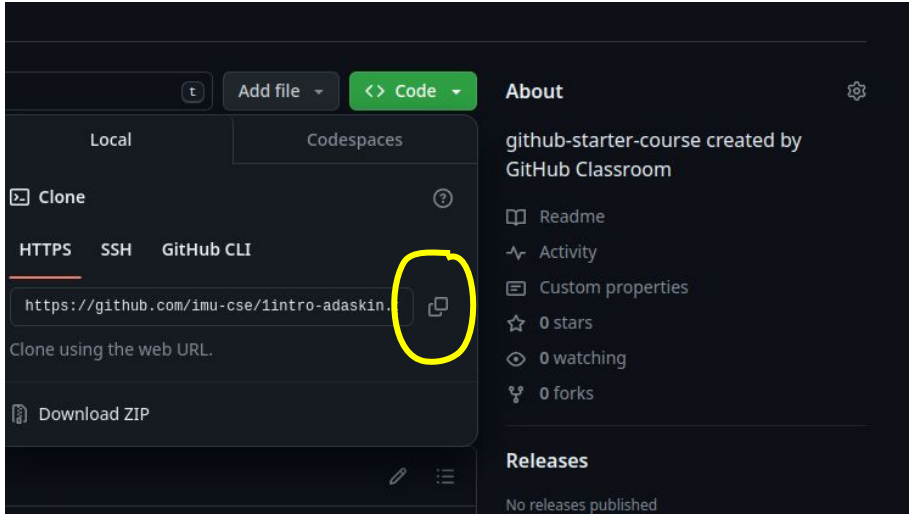
click the assignment to open on browser



```
$ git clone https://github.com/YOUR-USERNAME/YOUR-REPOSITORY
$ cd YOUR-REPOSITORY
```

Make some changes or add files

```
$ echo "int main(){}" > newfile.c
$ git add newfile.c
$ git commit -m "new file added!"
$ git push origin main
```



- You have pushed the changes to remote repo on github.comand submitted your assignment

For secure connection

Create ssh-key

https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent

And use gpg-key for signing commits

https://docs.github.com/en/authentication/managing-commit-signature-verification/adding-a-gpg-key-to-your-github-account

Crash course on C

see .c files posted on classroom

- **For C programming,** you can refer to "The C Programming Language", Brian W Kernighan, Dennis M Ritchie.
- C-Lecture notes from BIL121: https://sites.google.com/view/adaskin/teaching/c-programming-lecture-notes

Program files

A program file contains all the necessary information to load an application into memory and make it run.It includes:

- machine instructions
- initialized data
- List of library dependencies
- List of memory sections that the program will use
- List of undefined values in the executable that will be known until the program is loaded into memory.

Compiling a c program

```
$ cat main.c

#include <stdio.h>

int main() {

    printf("Hello World!\n");

    return 0;

}



$ gcc main.c -o main

$ ./main

Hello World!

$
```

Preprocessing:

- The preprocessor expands all preprocessor directives.

Parsing:

- The compiler parses the text file for function declarations, variable declarations, etc.

Assembly Generation:

- The compiler then generates assembly code for all the functions after some optimizations if enabled.

Assembler (compiling assembly code):

- The assembler turns the assembly into 0s and 1s and creates an object file. This object file maps names to pieces of code.

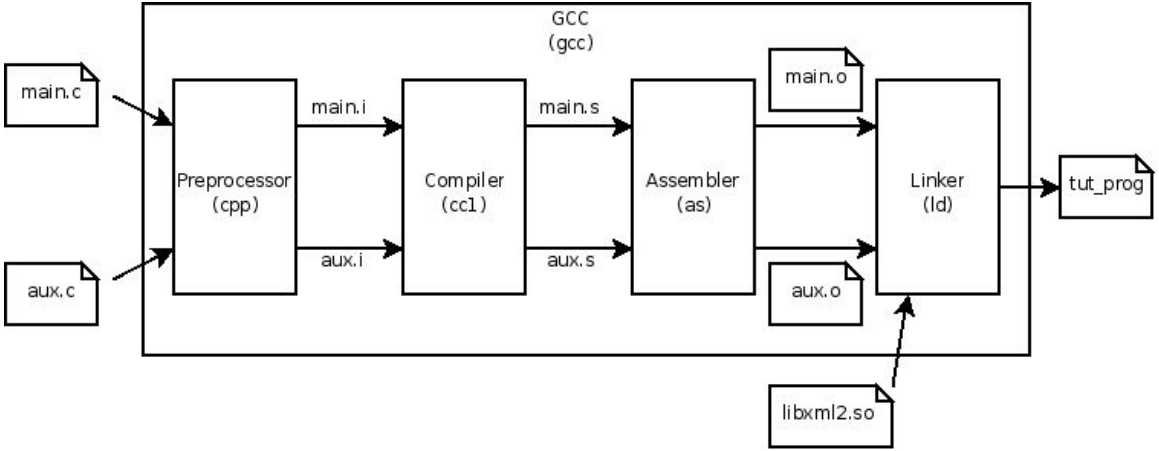
Static Linking:

- The linker takes a series of objects and static libraries and resolves references of variables and functions from one object file to another.
- The linker finds the main method and makes that the entry point for the function.
- The linker also notices when a function is meant to be dynamically linked.
- The compiler also creates a section in the executable that tells the operating system that these functions need addresses right before running.

Dynamic Linking: As the program is getting ready to be executed, the operating system looks at what libraries that the program needs and links those functions to the dynamic library.

- The program is run.

```
$ gcc main.c aux.c -o tut_prog
$ ./main
Hello World!
$
```



Img src: https://en.wikipedia.org/wiki/GNU_Compiler_Collection

```
$ cat main.c
#include <stdio.h>
int main() {
      printf("Hello World!\n");
      return 0;
}
```

```
$ gcc main.c -o main
$ ./main
Hello World!
$
```

- **The preprocessor**
  - expands **#define, #include, #ifdef** etc preprocessor statements and generates a **main.i** file.

- The compiler
  - compiles **main.i**, optimizes it and generates an assembly instruction: **main.s**

- The **assembler (as)**
  - takes input assembly file **main.s** and generates an object file **main.o**

- The compiler (cc or gcc) by default hides all these intermediate steps. You can use compiler options to run each step independently.

```
$ cat main.c
#include <stdio.h>
int main() {
      printf("Hello World!\n");
      return 0;
}
```

```
$ gcc main.c -o main
$ ./main
Hello World!
$
```

- `$ gcc -E main.c > main.i`
  - -E stops compiler after running preprocessor
- `$ gcc -S main.c`
  - -S stops compiler after assembly file generated
  - Assembler uses “.s” file to generate an object file
- `$ gcc -c main.c`
  - Generates object file main.o
  - main.o has undefined symbols like printf we don’t know where it is placed
  - `$ nm main.o`
- `$ gcc -o main main.c`
  - Generates executable main
  - printf does not have a value yet until the program is loaded
  - `$ nm main.o`

Gcc options
https://gcc.gnu.org/onlinedocs/gcc/Option-Summary.html#Option-Summary

Loading a program

- The loader is a program that is used to run an executable file in a process.
- Before the program starts running, the loader allocates space for all the sections of the executable file (text, data, bss etc)
- It loads into memory the executable and shared libraries (if not loaded yet)
- It also writes (resolves) any values in the executable to point to the functions/variables in the shared libraries.(E.g. calls to printf in main.c)
- Once memory image is ready, the loader jumps to the **_start** entry point that calls **init()** of all libraries and initializes static constructors. Then it calls **main()** and the program begins.
- **_start** also calls **exit()** when **main()** returns.
- The loader is also called "**runtime linker**".
- To see dependent shared libraries
  - `$ ldd a.out`

Static and Shared Libraries

- Shared libraries are shared across different processes.
  - .so files
  - Does not add to the size of executable
  - There is only one instance of each shared library for the entire system.
- Static libraries are not shared.
  - .a files
  - Add to the size of the executable
- There is an instance of an static library for each process.

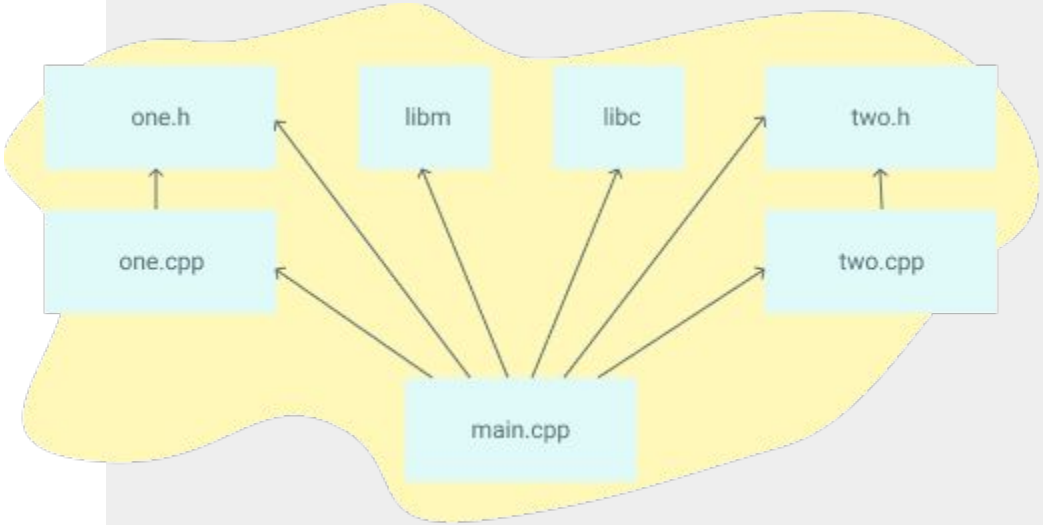
- Static or Compilation Time– Events that happen during program building.
  - Example: Static linker

- Dynamic or Runtime – Events that happen while program is running.
  - Example: Dynamic linker

Makefiles

https://makefiletutorial.com/

https://www.gnu.org/software/make/manual/make.html#Introduction



A simple makefile describes

- rules to compile and link source and header files
- and/or rules to recompile them

Alternatives to make

Popular C/C++ alternative build systems are SCons, CMake, Bazel, and Ninja.

Microsoft Visual Studio have their own built in build tools.

For Java, there's Ant, Maven, and Gradle.

Other languages like Go and Rust have their own build tools.

A simple Makefile

```
target … : prerequisites …
        recipe
        …
        …
```

prerequisite : An input file to create target

recipe : An action (a terminal command)

Makefile

```
hello:
     echo "Hello, World"
```

Run with make command

```
$ make
echo "Hello, World"
Hello, World
```

```
$ make hello
echo "Hello, World"
Hello, World
```

A simple Makefile

```
target … : prerequisites …
        recipe
        …
        …
```

Makefile

```
main:
     cc main.c -o program
```

Run with make command

```
$ make
```

```
$ make main
```

Multiple Target

```
target … : prerequisites …
        recipe
        …
        …
```

Makefile

```
main:
        cc main.c -o aprogram
run:
        ./aprogram
clean:
        rm aprogram
```

Run with make command

```
$ make
```

```
$ make main
cc main.c -o aprogram

$ make run
./aprogram
hello world!

$ make clean
rm aprogram
```

Multiple Target

```
target … : prerequisites …
        recipe
        …
        …
```

Makefile

```
some_file: other_file
        echo "This will always run, and runs second"
        touch some_file

other_file:
        echo "This will always run, and runs first"
```

```
$ make
echo "This will always run, and runs first"
This will always run, and runs first
echo "This will always run, and runs second"
This will always run, and runs second
touch some_file
```

Multiple Target

```
target … : prerequisites …
        recipe
        …
        …
```

```c
/*main.c*/
#include <stdio.h>
int printinteger(int);
int main(){
        printinteger(5);
        return 0;
}
```

```c
/*aux.c*/
#include <stdio.h>
int printinteger(int a){
        printf("integer: %d\n", a);
        return 0;
}
```

Makefile

```
main: aux.o main.c
        cc main.c aux.o -o aprogram

aux.o: aux.c
        cc -c aux.c
clean:
        rm -f aprogram
cleanall:
        rm -f aprogram *.o
```

Variables

```
/*main.c*/
#include <stdio.h>
int printinteger(int);
int main(){
        printinteger(5);
        return 0;
}
```

```
/*aux.c*/
#include <stdio.h>
int printinteger(int a){
        printf("integer: %d\n", a);
        return 0;
}
```

```
target … : prerequisites …
        recipe
        …
        …
```

Makefile

```
objects =  aux.o main.o

aprogram: $(objects)
        cc -o aprogram $(objects)

aux.o: aux.c
        cc -c aux.c

main.o: main.c
        cc -c main.c

clean:
        rm aprogram $(objects)
```

Variables

```
files := file1 file2
some_file: $(files)
	echo "Look at this
variable: " $(files)
	touch some_file

file1:
	touch file1
file2:
	touch file2

clean:
	rm -f file1 file2
some_file
```

Automatic variables

```
hey: one two
	# Outputs "hey", since this is the target name
	echo $@

	# Outputs all prerequisites newer than the target
	echo $?

	# Outputs all prerequisites
	echo $^

	touch hey

one:
	touch one

two:
	touch two

clean:
	rm -f hey one two
```

Fancy rules

Implicit Rules

```
CC = gcc # Flag for implicit rules
CFLAGS = -g # Flag for implicit rules. Turn on debug info

# Implicit rule #1: blah is built via
# the C linker implicit rule
# Implicit rule #2: blah.o is built via
#the C compilation implicit rule, because blah.c exists

blah: blah.o

blah.c:
	echo "int main() { return 0; }" > blah.c

clean:
	rm -f blah*
```

Static pattern rules

targets...: target-pattern: prereq-patterns ...

commands

```
objects = foo.o bar.o all.o
all: $(objects)

# These files compile via implicit rules
foo.o: foo.c
bar.o: bar.c
all.o: all.c

all.c:
	echo "int main() { return 0; }" > all.c

%.c:
	touch $@

clean:
	rm -f *.c *.o all
```

For more see https://makefiletutorial.com/

This week’s lab: Version control Git

This week’s lab assignment is given through github

- you will create multiple c files and makefile for a solution of simple program
- and submit it to github through git-commands from terminal

Next lecture

More on C, pointers, structures and alignments